



TRUSTSEC

Smart Contract Audit

Morpho – Midnight PR #1180

24/09/2026

Executive summary



Category	Lending
Audited file count	3
Auditor	Trust HollaDieWaldfee
Time period	17/09/2026 - 23/09/2026

Findings

Severity	Total	Acknowledged
High	0	0
Medium	0	0
Low	1	1

Centralization score



Signature

Digitally
signed by
Trust
Date:
2026.09.24
12:59:40
+03'00'



EXECUTIVE SUMMARY	1
DOCUMENT PROPERTIES	3
Versioning	3
Contact	3
INTRODUCTION	4
Scope	4
Repository details	4
About TrustSec	4
About the Auditors	4
Disclaimer	4
Methodology	5
QUALITATIVE ANALYSIS	6
FINDINGS	7
Low severity findings	7
TRST-L-1 Invalidating a root does not propagate to separately ratified sub-trees	7
Additional recommendations	8
TRST-R-1 <i>isRootRatified()</i> and <i>rootNonce()</i> can be declared external	8
TRST-R-2 Documentation for offer requirements is incomplete	8
TRST-R-3 Consider adding nonce and deadline protection to direct root updates	8
TRST-R-4 Document that <i>setIsRootRatified()</i> does not update <i>rootNonce</i>	9
TRST-R-5 Document that effective buyer price depends on continuous fee	9
Centralization risks	10
TRST-CR-1 Authorized addresses are fully trusted by the authorizer	10
Systemic risks	11
TRST-SR-1 Ratifiers depend on Morpho Midnight	11
TRST-SR-2 Chain outages can prevent offer fills	11
TRST-SR-3 <i>RateRatifierV1</i> uses simple instead of compounded interest	11

Document properties

Versioning

Version	Date	Description
0.1	23/09/2026	Client report
0.2	24/09/2026	Mitigation review

Contact

Trust

trust@trust-security.xyz

Introduction

TrustSec has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- `src/ratifiers/PriceRatifierV1.sol`
- `src/ratifiers/RateRatifierV1.sol`
- `src/ratifiers/libraries/HashLib.sol`

Repository details

- **Repository URL:** <https://github.com/morpho-org/midnight>
- **Commit hash:** 531ab3b23f95dcc660b8805e46f089248c6b994e
- **Mitigation hash:** bc8be43c7fbcd54831701a23270c42e950e96ed8

About TrustSec

TrustSec has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

Trust has established a dominating presence in the smart contract security ecosystem since 2022. He is a resident on the Immunefi, Sherlock and C4 leaderboards and is now focused in auditing and managing audit teams under TrustSec. When taking time off auditing & bug hunting, he enjoys assessing bounty contests in C4 as a Supreme Court judge.

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at TrustSec and Senior Watson at Sherlock.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. TrustSec assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	Excellent	The ratifiers are minimal and simple periphery contracts.
Documentation	Good	While the ratifiers are easy to understand, their documentation could be more detailed.
Best practices	Excellent	Project adheres to best practices.
Centralization risks	Excellent	The ratifiers don't introduce additional centralization risks.

Findings

Low severity findings

TRST-L-1 Invalidating a root does not propagate to separately ratified sub-trees

- **Category:** Logical issues
- **Source:** PriceRatifierV1.sol, RateRatifierV1.sol
- **Status:** Acknowledged

Description

Both ratifiers store the ratification status and nonce independently for each **(maker, root)** pair. Consequently, setting one root to **false** has no effect on another ratified root, even when the second root is a subtree containing the same offers.

This can occur when an offer tree is expanded by wrapping its previous root as a subtree. For example, a maker may first ratify **N = [A, B]** and later ratify **R = [[A, B], [C, D]]**. If the maker subsequently unratifies only **R**, offers **A** and **B** remain executable using proofs against **N** because **ratification[maker][N].isRootRatified** remains **true**.

As a result, a maker who believes that disabling the latest root disables every offer represented by that tree may still be filled through an older, independently ratified subtree, potentially executing stale or misconfigured offers.

Even if this is well understood by the dev team, without sufficient documentation of the behavior the issue could arise from 3rd-party integrations, which Morpho intends to support.

Recommended mitigation

Document that invalidation applies only to the exact root and that independently ratified overlapping roots must be disabled separately.

```
/// @dev Ratification status is scoped to the exact `(maker, root)` pair. Unratifying  
this root does not  
/// unratify other roots or subtrees containing the same offers. Independently  
ratified roots must be  
/// unratified separately.
```

For global invalidation, consider adding a per-maker epoch that can invalidate every previously ratified root at once.

Team response

Acknowledged. We think it's unnecessary to add further comments because it should be clear that unratifying a root doesn't unratify other roots regardless of whether they contain the same offers.

Additional recommendations

TRST-R-1 *isRootRatified()* and *rootNonce()* can be declared external

Since [*isRootRatified\(\)*](#) and [*rootNonce\(\)*](#) are not called internally, they can be declared **external** as a best practice. This recommendation applies to both ratifiers.

TRST-R-2 Documentation for offer requirements is incomplete

Both ratifiers include this comment:

```
/// @dev All offers in a tree are expected to share the same maker and ratifier.  
Otherwise all offers in a  
/// tree might not be ratified or unrated by a single call to either root setter.
```

It's also necessary that the ratifier in the offer is the ratifier in which the root is set. And the maker in the offer must be the maker for which the root is set. For example, the above comment still permits Alice to set a root with Bob's offers. The comment could be amended as follows:

```
/// @dev To control every offer in a tree with one root setting, every offer must use  
the same maker,  
/// that maker must be passed to the root setter, and every offer must set `ratifier`  
to this contract.
```

TRST-R-3 Consider adding nonce and deadline protection to direct root updates

Although the **nonce** and **deadline** in *setIsRootRatifiedWithSig()* primarily protect off-chain signatures, they could also benefit *setIsRootRatified()*:

- A deadline prevents a delayed transaction from changing a root after its intended validity window.
- An expected nonce prevents a stale transaction from overwriting a newer decision. For example, Bob's earlier activation transaction should not reactivate a root after Alice has disabled it upon discovering a misconfigured offer.
- Advancing **rootNonce** on direct updates would also invalidate outstanding signed updates for the previous state.

The expected single-writer model and bundler-level deadlines reduce these risks, but the contract does not enforce those assumptions. Consider accepting an expected nonce and deadline in the direct setter.

TRST-R-4 Document that *setIsRootRatified()* does not update rootNonce

It can be unexpected to users that *setIsRootRatified()* does not increment **rootNonce** since **rootNonce** can be understood as the number of times that either *setIsRootRatified()* or *setIsRootRatifiedWithSig()* has been called and changed state. Therefore, documentation that only *setIsRootRatifiedWithSig()* affects the **rootNonce** can avoid unnecessary issues.

TRST-R-5 Document that effective buyer price depends on continuous fee

The effective price paid by the buyer for a unit of credit depends not just on the tick of the offer but also on the continuous fee. It should be documented that the continuous fee is controlled separately via **offer.continuousFeeCap** and is not part of the **rate**.

Centralization risks

TRST-CR-1 Authorized addresses are fully trusted by the authorizer

Users can grant permissions to act on their behalf through Midnight's [isAuthorized](#) mapping. An authorized address has full control over the authorizer's Midnight account and can ratify roots on behalf of the authorizer.

Systemic risks

TRST-SR-1 Ratifiers depend on Morpho Midnight

The audited ratifiers are periphery contracts for Morpho Midnight. An exploit, accounting failure, or loss of availability in any dependency can expose users to losses. Ratifiers cannot isolate users from failures in those contracts. Moreover, Midnight markets that users interact with must satisfy the safety conditions that are documented in Midnight, for example supported token assumptions.

TRST-SR-2 Chain outages can prevent offer fills

Ratifiers allow users to act as passive makers and post offers that takers can fill. Even if takers are incentivized to take the posted offers, a chain outage can prevent execution. Once the chain resumes, users may not be able to manage their position before, for example, they are liquidated.

TRST-SR-3 *RateRatifierV1* uses simple instead of compounded interest

RateRatifierV1 converts a maker's signed per-second rate into a price bound of $1 / (1 + \text{rate} * \text{timeToMaturity})$ using the money market quoting method. While this is documented in the [NatSpec](#), it must be understood that a compounded per-second rate over **timeToMaturity** is higher than a simple rate. The error increases as **timeToMaturity** and the **rate** increase. E.g. a per-second rate of **10%/1year** would result in roughly **10.5%** when compounded each second. Users must be aware of the effect of using simple interest and set their ratifications accordingly.